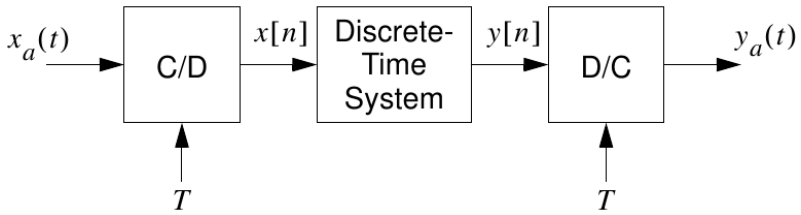
Chapter 8
Digital Filter Design

Contents

Introduction

- A digital filter is a frequency-selective LTI system that passes selected frequency components and rejects others.
- The discrete-time realizations of interest are causal systems with LCCDE models.
- Classical analog approximations provide the main foundation for the digital filter-design methods in this chapter.
- The design problem is naturally split into three stages:
  - specification of desired system properties,
  - approximation of those specifications with causal discrete-time systems, and
  - system realization in hardware or software.

Introduction



- A common use case is continuous-time filtering using an A/D–$H(z)$–D/A chain; strictly speaking $H(z)$ is a discrete-time filter, although it is commonly called a digital filter.

8.1 Overview of Approximation Techniques

Continuous-time to discrete-time specification mapping

$$H_{\text{eff}}(j\Omega) = \begin{cases} H\left(e^{j\Omega T}\right), & |\Omega| < \pi/T, \\ 0, & |\Omega| \geq \pi/T, \end{cases} \qquad H(e^{j\omega}) = H_{\text{eff}}\left(j\frac{\omega}{T}\right), \quad |\omega| < \pi.$$

- Digital filter-design methods fall into either IIR or FIR approaches.
- Typical IIR approaches: impulse invariance, bilinear transformation, and frequency-domain least-squares design, and numerical solution of differential equations.
- Typical FIR approaches: truncated impulse responses with windows, frequency sampling, equiripple design, and frequency-domain least-squares design.

8.2 Continuous-Time Filter Design Overview

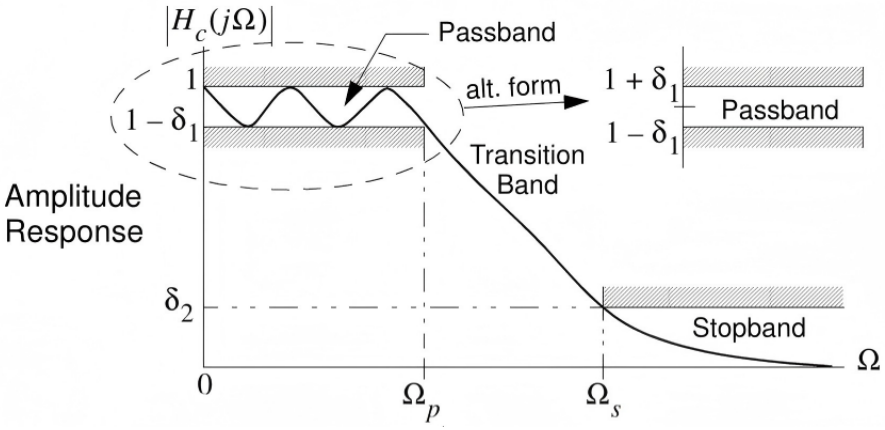
General continuous-time prototype

$$H_c(s) = \frac{\sum_{m=0}^{M_c} c_m s^m}{\sum_{k=0}^{N} d_k s^k}, \qquad M_c \leq N,$$

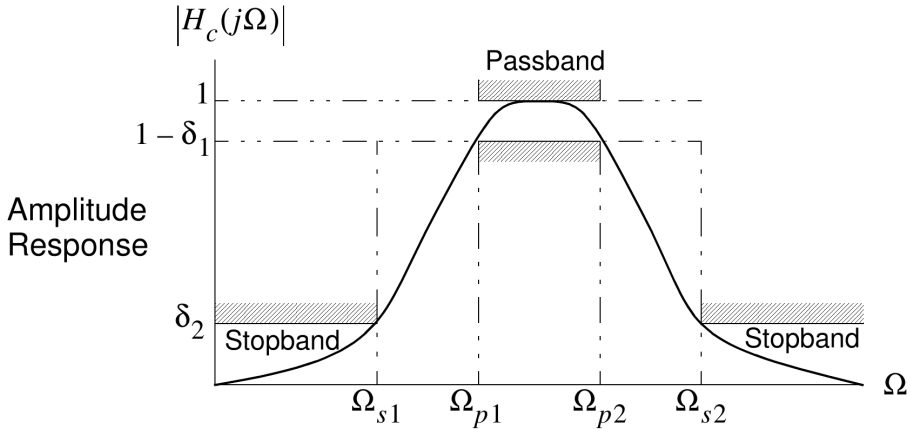
so that the gain remains finite as $\Omega \to \infty$.

- The classical analog families used here are Butterworth (maximally flat), Chebyshev type I, Chebyshev type II, and elliptic filters.
- Specifications are usually expressed in terms of amplitude, phase, group delay, or a combination of amplitude and phase.
- The focus in this chapter is primarily on amplitude-response approximation.

Example 8.1 – Lowpass Specifications



Example 8.2 – Bandpass Amplitude Specifications



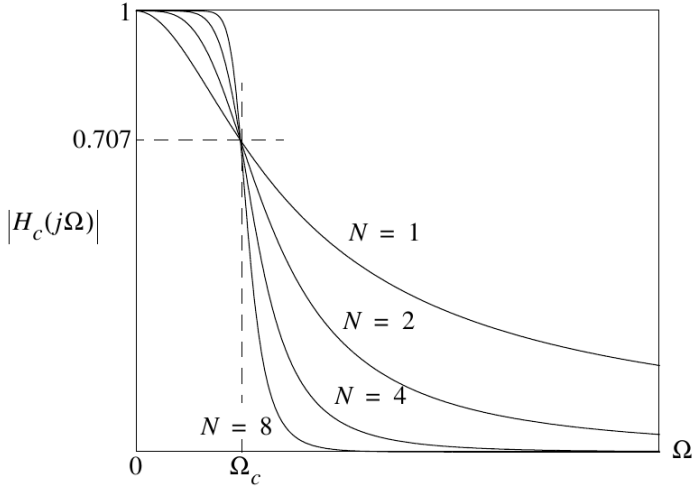
8.3 Butterworth Design – Lowpass Specifications

- The filter requirements are stated in terms of amplitude response.
- For lowpass design the standard constraints are

$$1 \geq |H_c(j\Omega)| \geq 1 - \delta_1, \quad |\Omega| \leq \Omega_p, \qquad |H_c(j\Omega)| \leq \delta_2, \quad |\Omega| \geq \Omega_s.$$

- Later, lowpass prototypes are transformed into highpass, bandpass, and bandstop filters.
- Butterworth filters are designed to maintain a smooth, monotone amplitude characteristic in both passband and stopband.

Butterworth Magnitude Response (Orders 1, 2, 4, 8)



8.3 Butterworth Design – Magnitude Response and Properties

Magnitude-squared transfer function

$$|H_c(j\Omega)|^2 = \frac{1}{1 + (\Omega/\Omega_c)^{2N}}.$$

- $|H_c(j\Omega)| = 1$ at $\Omega = 0$ for every order N.
- At the cutoff, $|H_c(j\Omega_c)| = 1/\sqrt{2}$, so Ω_c is the familiar 3 dB frequency.
- The response is monotone decreasing and approaches the ideal lowpass characteristic as $N \to \infty$.
- Above cutoff the rolloff is $20N$ dB/decade, or equivalently $6N$ dB/octave.

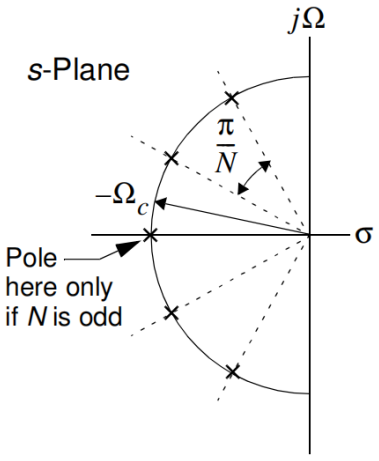
8.3 Butterworth Design – System Function and Pole Pattern

Pole-factor form

$$H_c(s) = \frac{1}{B_N(s)} = \frac{1}{\prod_{k=1}^{N}(s - s_k)}, \qquad s_k = \Omega_c e^{j\pi[0.5-(2k-1)/(2N)]}.$$

- A Butterworth lowpass has N zeros at infinity.
- If N is odd, one pole lies on the negative real axis at $-\Omega_c$.
- Standard-form and quadratic-factored Butterworth polynomials are tabulated for normalized cutoff $\Omega_c = 1$.
- Frequency scaling to a new cutoff is performed by the substitution $s \mapsto s/\Omega_c$.

Butterworth Pole Locations



Butterworth pole locations.

8.3 Butterworth Polynomials for $\Omega_c = 1$

$$
\begin{aligned}
B_1(s) &= s + 1, \\
B_2(s) &= s^2 + \sqrt{2}\, s + 1, \\
B_3(s) &= (s^2 + s + 1)(s + 1) = s^3 + 2s^2 + 2s + 1, \\
B_4(s) &= (s^2 + 0.76536s + 1)(s^2 + 1.84776s + 1), \\
B_5(s) &= (s + 1)(s^2 + 0.6180s + 1)(s^2 + 1.6180s + 1), \\
B_6(s) &= (s^2 + 0.5176s + 1)(s^2 + \sqrt{2}\, s + 1)(s^2 + 1.9318s + 1).
\end{aligned}
$$

- The normalized factored form is often the most convenient representation for implementation and scaling.
- Frequency scaling to a new cutoff is obtained by the substitution $s \mapsto s/\Omega_c$.

Example 8.3 – Obtaining the Butterworth Polynomial

- Design a Butterworth lowpass with $\Omega_c = \Omega_p = 1$ rad/s, $\Omega_s = 4$ rad/s, and $20\log_{10}\delta_2 = -24$ dB.
- Since Ω_s is two octaves above Ω_c, the required rolloff is 12 dB/octave, so order $N \geq 2$ is sufficient.
- From the normalized Butterworth table,

$$H_c(s) = \frac{1}{s^2 + \sqrt{2}\,s + 1}.$$

- This shows how the lookup tables are used before general amplitude-specification formulas are introduced.

8.3.1 General Butterworth Design from Amplitude Specifications

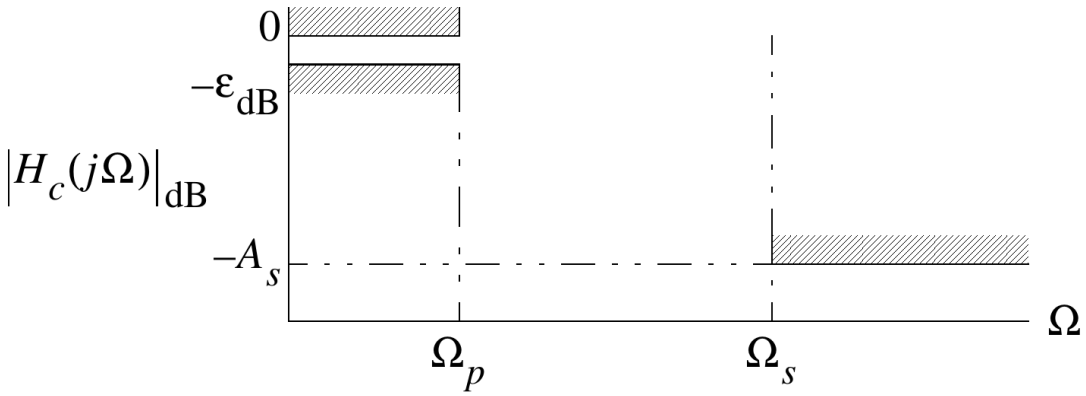
Given the lowpass amplitude specifications

$$-\epsilon_{dB} \leq 20\log_{10}|H_c(j\Omega)| \leq 0, \qquad |\Omega| \leq \Omega_p$$

$$20\log_{10}|H_c(j\Omega)| \leq -A_s, \qquad |\Omega| \geq \Omega_s$$

we wish to determine the Butterworth filter order N and cutoff frequency Ω_c.

8.3.1 Butterworth Design – Amplitude Constraints



Amplitude-response constraints.

8.3.1 General Butterworth Design from Amplitude Specifications

For a Butterworth lowpass filter,

$$|H_c(j\Omega)|^2 = \frac{1}{1+\left(\frac{\Omega}{\Omega_c}\right)^{2N}}.$$

Thus the passband and stopband equalities are written as

$$10\log_{10}\left[\frac{1}{1+\left(\frac{\Omega_p}{\Omega_c}\right)^{2N}}\right] = -\epsilon_{dB},$$

$$10\log_{10}\left[\frac{1}{1+\left(\frac{\Omega_s}{\Omega_c}\right)^{2N}}\right] = -A_s.$$

8.3.1 Solving for N and Ω_c

Rewriting the two design equations gives

$$10^{\epsilon_{dB}/10} - 1 = \left(\frac{\Omega_p}{\Omega_c}\right)^{2N}, \qquad 10^{A_s/10} - 1 = \left(\frac{\Omega_s}{\Omega_c}\right)^{2N}.$$

Dividing the first equation by the second yields

$$\left(\frac{\Omega_p}{\Omega_s}\right)^{2N} = \frac{10^{\epsilon_{dB}/10} - 1}{10^{A_s/10} - 1}.$$

Hence the required filter order is

$$N = \left\lceil \frac{\log_{10}\left(\dfrac{10^{\epsilon_{dB}/10} - 1}{10^{A_s/10} - 1}\right)}{2\log_{10}(\Omega_p/\Omega_s)} \right\rceil.$$

Here $\lceil \cdot \rceil$ denotes the ceiling operator.

8.3.1 Choice of Ω_c

Since N must be an integer, the passband and stopband constraints will not in general both be met with equality.

- If equality is enforced at the passband edge Ω_p (most common), choose

$$\Omega_c = \frac{\Omega_p}{\left(10^{\epsilon_{dB}/10} - 1\right)^{1/(2N)}}.$$

- If equality is enforced at the stopband edge Ω_s, choose

$$\Omega_c = \frac{\Omega_s}{\left(10^{A_s/10} - 1\right)^{1/(2N)}}.$$

- A third possibility is to choose Ω_c between these two values so that both requirements are exceeded.

  With the usual positive dB definitions,

$$\epsilon_{dB} = -20\log_{10}(1-\delta_1), \qquad A_s = -20\log_{10}(\delta_2).$$

Example 8.4: A Butterworth Amplitude Response Design

Let

$$\Omega_p = 20 \text{ rad/s}, \qquad \epsilon_{dB} = 2 \text{ dB},$$
$$\Omega_s = 30 \text{ rad/s}, \qquad A_s = 10 \text{ dB}.$$

Then

$$N = \left\lceil \frac{\log_{10}\left(\dfrac{10^{2/10}-1}{10^{10/10}-1}\right)}{2\log_{10}(20/30)} \right\rceil = \lceil 3.3709 \rceil = 4.$$

Matching at Ω_p gives

$$\Omega_c = \frac{20}{\left(10^{2/10}-1\right)^{1/8}} = 21.3868 \text{ rad/s}.$$

Example 8.4: Frequency-Scaled Transfer Function

From the normalized Butterworth table, the fourth-order prototype is

$$H_c(s) = \frac{1}{(s^2 + 0.76536s + 1)(s^2 + 1.84776s + 1)}.$$

To scale from $\Omega_c = 1$ to $\Omega_c = 21.3868$, let

$$s \to \frac{s}{21.3868}.$$

Hence the frequency-scaled transfer function becomes

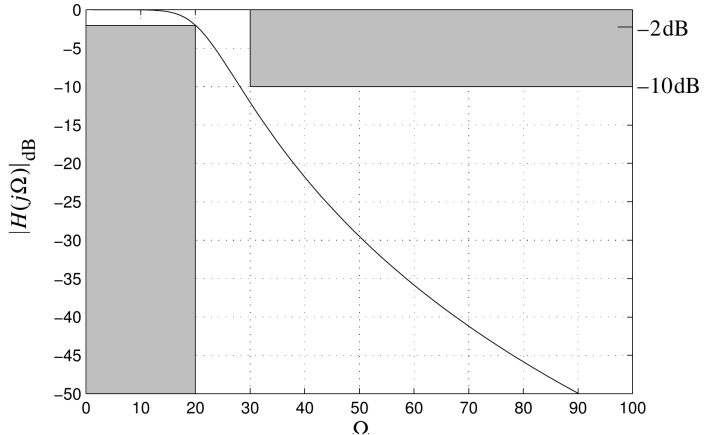
$$H_c(s) = \frac{(21.3868)^4}{(s^2 + 16.37s + 457.4)(s^2 + 39.52s + 457.4)}.$$

Example 8.4 – MATLAB Analysis Commands

We can use MATLAB to directly analyze the above Butterworth design.

```
w = 0.1:100;
H = freqs(1,conv([1 .76536 1],[1 1.8477 1]),w/21.3868);
plot(w,20*log10(abs(H)))
grid
axis([0 100 -50 0])
patch([0 20 20 0],[-50 -50 -2 -2],[.75 .75 .75])
patch([30 100 100 30],[-10 -10 0 0],[.75 .75 .75])
```

Example 8.4 – Butterworth Magnitude Response



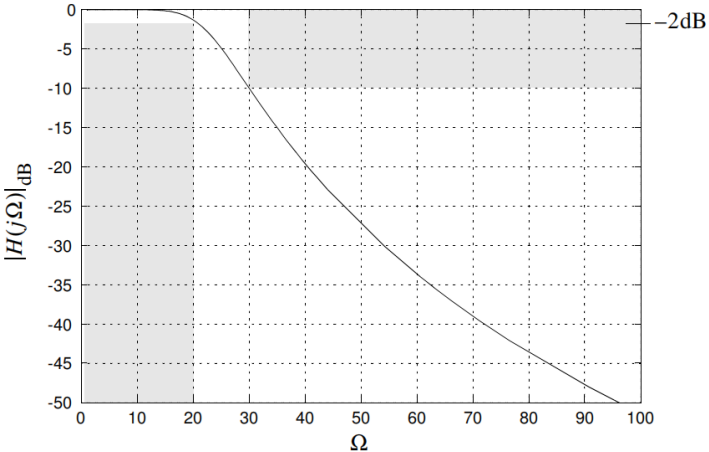
Butterworth magnitude response.

Example 8.4 – All-MATLAB Butterworth Design

An alternative approach is to design the filter completely using MATLAB filter design tools for the *s*-domain

```
[n,Wn] = buttord(20,30,2,10,'s')
[b,a] = butter(n,Wn,'s')
[H,w] = freqs(b,a);
plot(w,20*log10(abs(H)));
```

Example 8.4 – All-MATLAB Butterworth Response

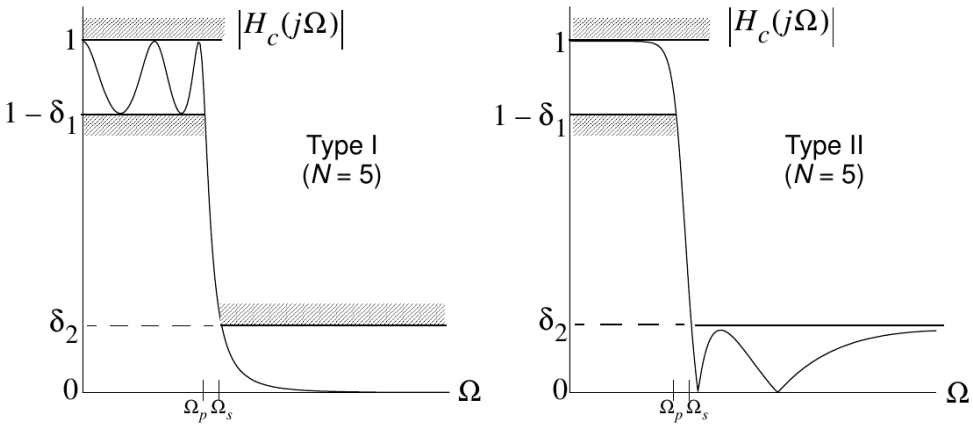


Butterworth magnitude response in the all-MATLAB design.

8.4 Chebyshev Design – Type I versus Type II

- Chebyshev filters trade monotone behavior for sharper rolloff near the cutoff frequency.
- Type I permits equiripple behavior in the passband while keeping the stopband monotone.
- Type II keeps the passband monotone and moves the ripple into the stopband.
- Both families achieve a more selective transition than Butterworth filters of the same order.

Chebyshev Type I versus Type II Responses



8.4.1 Chebyshev Type I

Magnitude response

$$|H_c(j\Omega)|^2 = \frac{1}{1+\epsilon^2 T_N^2(\Omega/\Omega_c)},$$

where $T_N(x)$ is the Nth-order Chebyshev polynomial.

- The recursion is $T_N(x) = 2xT_{N-1}(x) - T_{N-2}(x)$ with $T_0(x) = 1$ and $T_1(x) = x$.
- A useful closed form is

$$T_N(x) = \begin{cases} \cos(N\cos^{-1}x), & |x| \le 1, \\ \cosh(N\cosh^{-1}x), & |x| > 1. \end{cases}$$

- In the passband, $1/\sqrt{1+\epsilon^2} \le |H_c(j\Omega)| \le 1$; the ripple in dB is $\epsilon_{\mathrm{dB}} = 10\log_{10}(1+\epsilon^2)$.

8.4.1 Chebyshev Type I – System Function

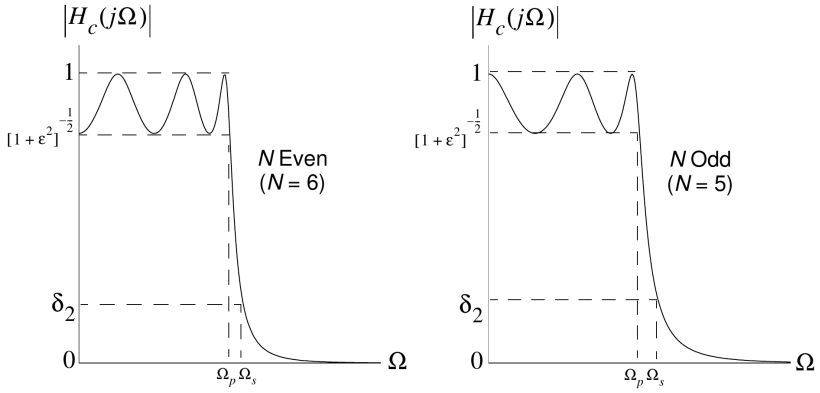
Pole-factor form

$$H_c(s) = \frac{K}{V_N(s)}, \qquad V_N(s) = \prod_{k=1}^{N} (s - s_k),$$

with poles located on an ellipse in the s-plane.

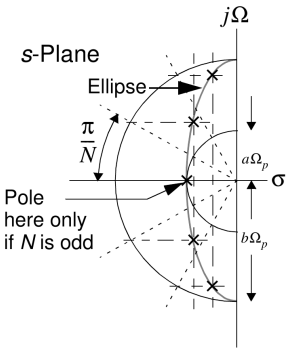
- The ellipse parameters are set by $\alpha = \epsilon^{-1} + \sqrt{1 + \epsilon^{-2}}$, $a = \frac{1}{2}(\alpha^{1/N} - \alpha^{-1/N})$, and $b = \frac{1}{2}(\alpha^{1/N} + \alpha^{-1/N})$.
- The gain factor K is chosen so that $H_c(0) = 1$ for odd N and $H_c(0) = 1/\sqrt{1 + \epsilon^2}$ for even N.
- Normalized Chebyshev polynomials $V_N(s)$ are tabulated for 0.5, 1, and 2 dB ripple.

Chebyshev Type I – Even and Odd Passband Ripple Behavior



The number of inflections indicates N.

Chebyshev Type I Pole Locations on an Ellipse



Chebyshev type I pole locations

8.4.1 Normalized Chebyshev Polynomials – $\epsilon_{\mathrm{dB}} = 0.5$

With $\epsilon = 0.34931$:

$$V_1(s) = s + 2.86277,$$
$$V_2(s) = s^2 + 1.42562s + 1.51620,$$
$$V_3(s) = s^3 + 1.25291s^2 + 1.53489s + 0.71570,$$
$$V_4(s) = s^4 + 1.19739s^3 + 1.71687s^2 + 1.02545s + 0.37905,$$
$$V_5(s) = s^5 + 1.17249s^4 + 1.93737s^3 + 1.30957s^2 + 0.75252s + 0.17892,$$
$$V_6(s) = s^6 + 1.15918s^5 + 2.17184s^4 + 1.58976s^3 + 1.17186s^2 + 0.43237s + 0.09476.$$

8.4.1 Normalized Chebyshev Polynomials – $\epsilon_{\mathrm{dB}} = 1$

With $\epsilon = 0.50885$:

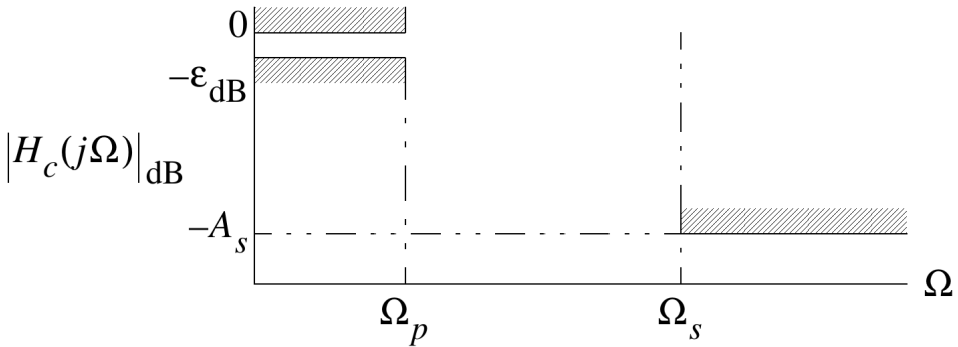
$$V_1(s) = s + 1.96523,$$
$$V_2(s) = s^2 + 1.09773s + 1.10251,$$
$$V_3(s) = s^3 + 0.98834s^2 + 1.23841s + 0.49131,$$
$$V_4(s) = s^4 + 0.95281s^3 + 1.45392s^2 + 0.74262s + 0.27563,$$
$$V_5(s) = s^5 + 0.93682s^4 + 1.68881s^3 + 0.97430s^2 + 0.58053s + 0.12283,$$
$$V_6(s) = s^6 + 0.92825s^5 + 1.93082s^4 + 1.20214s^3 + 0.93935s^2 + 0.30708s + 0.06891.$$

8.4.1 Normalized Chebyshev Polynomials – $\epsilon_{\mathrm{dB}} = 2$

With $\epsilon = 0.76478$:

$$V_1(s) = s + 1.30756,$$
$$V_2(s) = s^2 + 0.80382s + 0.63677,$$
$$V_3(s) = s^3 + 0.73782s^2 + 1.02219s + 0.32689,$$
$$V_4(s) = s^4 + 0.71621s^3 + 1.25648s^2 + 0.51680s + 0.20576,$$
$$V_5(s) = s^5 + 0.70646s^4 + 1.49954s^3 + 0.69348s^2 + 0.45935s + 0.08172,$$
$$V_6(s) = s^6 + 0.70123s^5 + 1.74586s^4 + 0.86701s^3 + 0.77146s^2 + 0.21027s + 0.05144.$$

8.4.2 General Chebyshev Type I Design from Specifications



Given ϵ_{dB}, A_s, Ω_p, and Ω_s, find N.

8.4.2 General Chebyshev Type I Design from Specifications

Order formula

Given ϵ_{dB}, A_s, Ω_p, and Ω_s,

$$N = \left\lceil \frac{\cosh^{-1}\left(\sqrt{\dfrac{10^{A_s/10}-1}{10^{\epsilon_{\mathrm{dB}}/10}-1}}\right)}{\cosh^{-1}(\Omega_s/\Omega_p)} \right\rceil .$$

- The design is driven by the desired stopband attenuation at Ω_s.
- Once the order is fixed, the tabulated or MATLAB-generated normalized polynomial is frequency scaled to the desired passband edge.

Example 8.5 – Chebyshev Type I Design

- Specifications: $\epsilon_{\mathrm{dB}} = 2$ dB, $A_s = 20$ dB, $\Omega_p = 40$ rad/s, $\Omega_s = 52$ rad/s.
- The order is

$$N = \left\lceil \frac{\cosh^{-1}\left(\sqrt{(10^{20/10}-1)/(10^{2/10}-1)}\right)}{\cosh^{-1}(52/40)} \right\rceil = 5.$$

- Using the 2 dB table,

$$H_c(s) = \left. \frac{0.0817}{s^5 + 0.706s^4 + 1.50s^3 + 0.694s^2 + 0.459s + 0.0817} \right|_{s \mapsto s/40}.$$

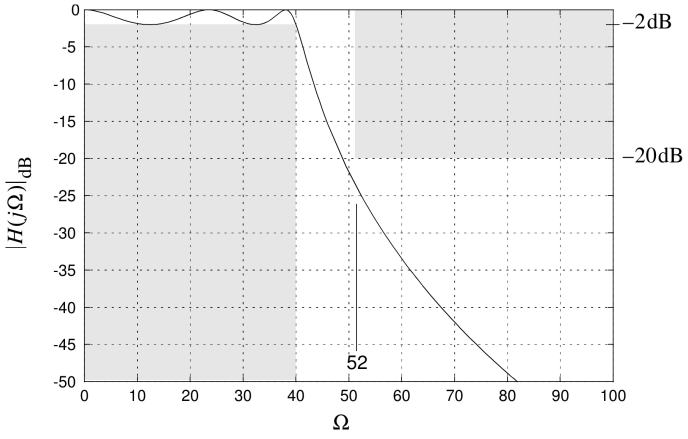
- MATLAB verification and synthesis use `freqs`, `cheb1ord`, and `cheby1`.

Example 8.5 – MATLAB Commands for the Type I Design

We can use MATLAB to directly analyze the above Chebyshev type I design

```
w = 0:100/200:100;
H = freqs(0.08172,[1 .70646 1.4995 .6935 .4593 .08172],w/40);
plot(w,20*log10(abs(H)));
axis([0 100 -50 0]);
grid;
```

Example 8.5 – Chebyshev Type I Magnitude Response



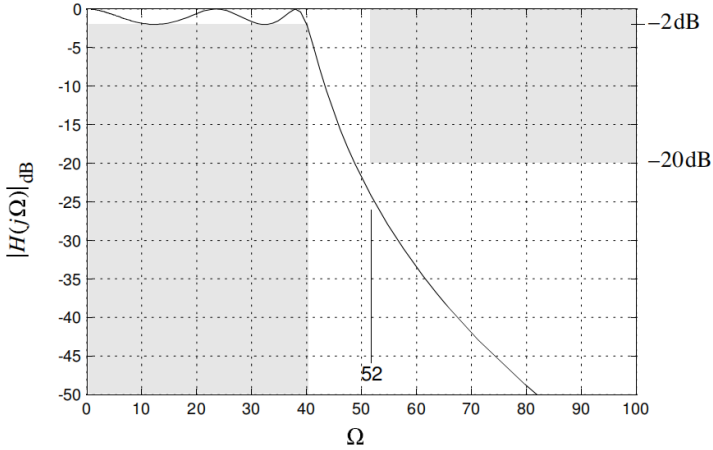
Chebyshev type I magnitude response.

Example 8.5 – MATLAB Commands for the Type I Design

An alternative approach is to design the filter completely using MATLAB filter design tools for the s-domain.

```
[n,Wn] = cheb1ord(40,52,2,20,'s');
[b,a]  = cheby1(n,2,Wn,'s');
[H,w] = freqs(b,a);
plot(w,20*log10(abs(H)));
```

Example 8.5 – All-MATLAB Chebyshev Type I Response



Chebyshev type I magnitude response in the all-MATLAB design.

8.4.4 Chebyshev Type II

Magnitude response

$$|H_c(j\Omega)|^2 = \frac{1}{1 + \epsilon^2 \dfrac{T_N^2(\Omega_s/\Omega_p)}{T_N^2(\Omega_s/\Omega)}}.$$

- The passband remains monotone and the stopband becomes equiripple.
- The passband ripple is still determined by ϵ_{dB} because $|H_c(j\Omega_p)|^2 = 1/(1+\epsilon^2)$.
- The system function has zeros on the $j\Omega$ axis and poles obtained from the same hyperbolic construction used for Type I.
- The order formula is the same as for Type I when written in terms of Ω_s/Ω_p.

8.4.4 Chebyshev Type II – System Function and Design Parameters

System function

$$H_c(s) = \frac{\prod_{m=1}^{N}(s - z_m)}{\prod_{k=1}^{N}(s - p_k)}, \qquad z_m = \frac{j\Omega_s}{\cos\left[\frac{\pi}{2N}(2m-1)\right]}.$$

For the poles, write $p_k = \alpha_k + j\beta_k$ with

$$\alpha_k = \frac{\Omega_p \Omega_s \sigma_k}{\sigma_k^2 + \omega_k^2}, \qquad \beta_k = \frac{\Omega_p \Omega_s \omega_k}{\sigma_k^2 + \omega_k^2},$$

and

$$\sigma_k = -\sin\left(\frac{\pi}{2N}(2k-1)\right)\sinh\left[\frac{1}{N}\sinh^{-1}\left(\frac{1}{\epsilon}\right)\right],$$

$$\omega_k = \cos\left(\frac{\pi}{2N}(2k-1)\right)\cosh\left[\frac{1}{N}\sinh^{-1}\left(\frac{1}{\epsilon}\right)\right].$$

Example 8.6 – Chebyshev Type II Design Setup

- Specifications: $\epsilon_{\mathrm{dB}} = 2$ dB, $A_s = 40$ dB, $\Omega_p = 100$ rad/s, and $\Omega_s = 200$ rad/s.
- The order formula is the same one used for Chebyshev type I designs:

$$N = \left\lceil \frac{\cosh^{-1}\left(\sqrt{\dfrac{10^{A_s/10}-1}{10^{\epsilon_{\mathrm{dB}}/10}-1}}\right)}{\cosh^{-1}(\Omega_s/\Omega_p)} \right\rceil = 5.$$

- The hand analysis keeps ϵ fixed, while MATLAB may adjust ϵ and the cutoff so that the exact minimum stopband attenuation equals the desired value.

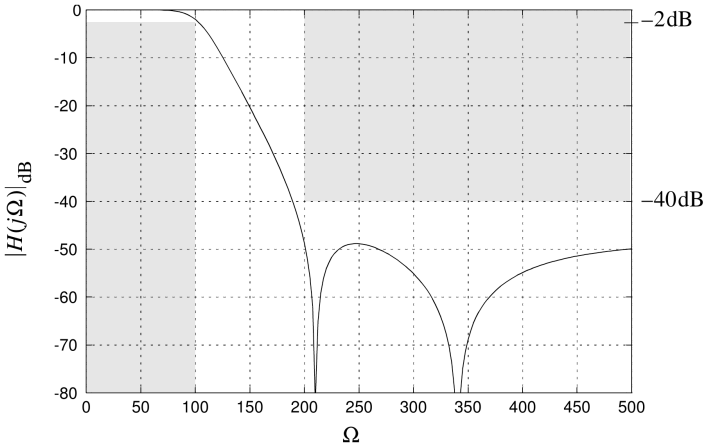
Example 8.6 – Chebyshev Type II Design

- Specifications: $\epsilon_{\mathrm{dB}} = 2$ dB, $A_s = 40$ dB, $\Omega_p = 100$ rad/s, $\Omega_s = 200$ rad/s.
- The order is $N = 5$.
- For analysis, the notes evaluate the Chebyshev polynomial with a helper MATLAB function `chebpoly`. MATLAB synthesis uses `cheb2ord` and `cheby2`.
- The MATLAB design adjusts ϵ and the cutoff slightly so that the minimum stopband attenuation is exactly 40 dB rather than the larger value obtained from the rounded-up hand design.

Example 8.6 – MATLAB Helper for $T_N(\Omega)$

```
function Tn = chebpoly(N,x)
Tn = zeros(size(x));
s0 = find(abs(x) < 1);
s1 = find(abs(x) >= 1);
Tn(s0) = cos(N*acos(x(s0)));
Tn(s1) = cosh(N*acosh(x(s1)));

w = 0:500/200:500;
H = -10*log10(1 + (10^(2/10)-1) ...
    *(chebpoly(5,2)./chebpoly(5,200./w)).^2);
plot(w,H); grid; axis([0 500 -80 0])
```

Example 8.6 – Hand / Polynomial Evaluation Plot

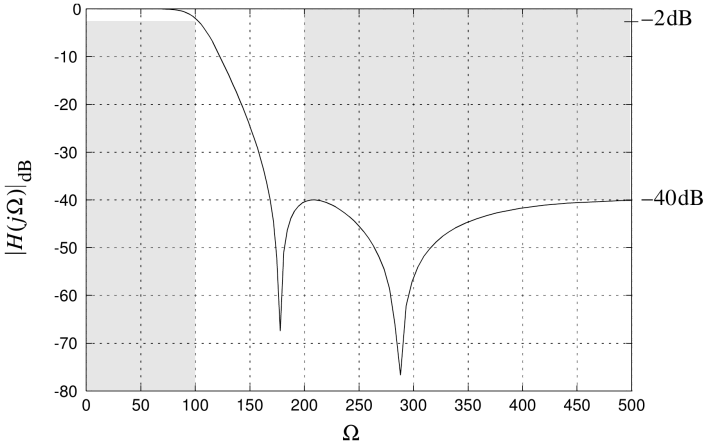


Hand / polynomial-evaluation magnitude response.

Example 8.6 – All-MATLAB Type II Design

```
[n,Wn] = cheb2ord(100,200,2,40,'s')
[b,a]  = cheby2(n,40,Wn,'s')
[H,w]  = freqs(b,a);
plot(w,20*log10(abs(H)));
```

Example 8.6 – All-MATLAB Type II Response



Chebyshev type II magnitude response in the all-MATLAB design.

8.5 Elliptic Design

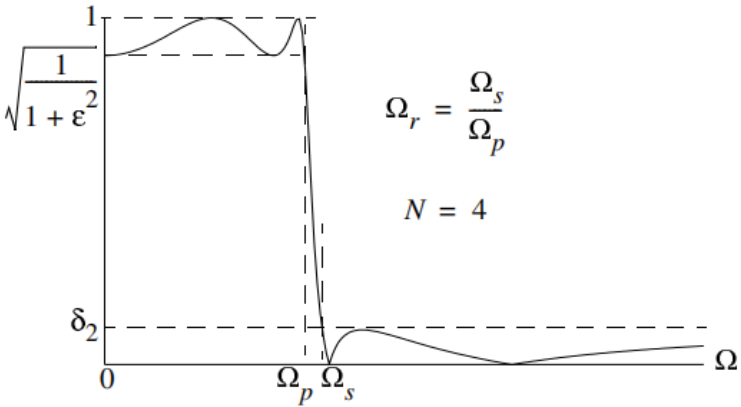
- Elliptic filters allow ripple in both the passband and the stopband in order to obtain the narrowest transition band for a given order.
- For the same filter order, no other classical approximation produces a sharper transition region.

Magnitude response

$$|H_c(j\Omega)|^2 = \frac{1}{1+\epsilon^2 U_N^2(\Omega/\Omega_p)}, \qquad \Omega_r = \frac{\Omega_s}{\Omega_p},$$

where $U_N(\cdot)$ is a Jacobian elliptic function.

8.5 Elliptic Design – Generic Magnitude Response



Elliptic generic magnitude response.

8.5 Elliptic Design

- Define the transition region ratio as

$$\Omega_r = \frac{\Omega_s}{\Omega_p}.$$

- The normalized lowpass system function can be written in factored form as

$$H_c(s) = \begin{cases} \dfrac{H_0}{s+s_0} \displaystyle\prod_{i=1}^{(N-1)/2} \dfrac{s^2 + A_{0i}}{s^2 + B_{1i}s + B_{0i}}, & N \text{ odd}, \\ \displaystyle\prod_{i=1}^{N/2} \dfrac{s^2 + A_{0i}}{s^2 + B_{1i}s + B_{0i}}, & N \text{ even}. \end{cases}$$

Note: $H_c(s)$ contains conjugate pairs of zeros on the $j\Omega$-axis which give the stopband nulls.

8.5 Elliptic Design

- The filter coefficients H_0, and s_0 if N is odd, and A_{0i}, B_{1i}, and B_{0i} are determined from the filter specifications ϵ_{dB}, A_s, and Ω_r.
- The following pages contain several of the design data tables which give the achieved value for Ω_r as a function of N for fixed ϵ_{dB} and A_s.

8.5 Elliptic Design

Passband Ripple $\epsilon_{\text{dB}} = 1$, Stopband Attenuation $A_s = 20$ dB

N	$H(s) = \dfrac{b_N s^N + b_{N-1} s^{N-1} + \ldots + b_1 s + b_0}{a_N s^N + a_{N-1} s^{N-1} + \ldots + a_1 s + a_0}$						$\dfrac{\Omega_s}{\Omega_p}$
2	$b_{N,\ldots,0}$			0.10001	0	1.02771	2.3240
	$a_{N,\ldots,0}$			1.00000	1.02957	1.15311	
3	$b_{N,\ldots,0}$		0	0.32057	0	0.66468	1.3078
	$a_{N,\ldots,0}$		1.00000	0.96658	1.24066	0.66468	
4	$b_{N,\ldots,0}$	0.09998	0	0.54212	0	0.52554	1.0902
	$a_{N,\ldots,0}$	1.00000	0.90376	1.67646	0.86878	0.58967	
5	$b_{N,\ldots,0}$	0.28302	0	0.74704	0	0 0.47648	1.0282
	$a_{N,\ldots,0}$	0.92728	2.04748	1.40558	1.03362	1.00000 0.47648	
6	$b_{N,\ldots,0}$	0.60190	0	0.95505	0.09996 0	0 0.45419	1.0090
	$a_{N,\ldots,0}$	2.57830	1.68592	2.08694	1.00000 0.79228	0.89292 0.50961	

8.5 Elliptic Design

Passband Ripple $\epsilon_{\text{dB}} = 1$, Stopband Attenuation $A_s = 30$ dB

N	$H(s) = \frac{b_N s^N + b_{N-1} s^{N-1} + \ldots + b_1 s + b_0}{a_N s^N + a_{N-1} s^{N-1} + \ldots + a_1 s + a_0}$						$\frac{\Omega_s}{\Omega_p}$
2	$b_{N,\ldots,0}$			0.03164	0	0.99795	4.0036
	$a_{N,\ldots,0}$			1.00000	1.07759	1.11971	
3	$b_{N,\ldots,0}$		0	0.14902	0	0.56866	1.7324
	$a_{N,\ldots,0}$		1.00000	0.97015	1.24597	0.56866	
4	$b_{N,\ldots,0}$	0.03163	0	0.28089	0	0.38968	1.2504
	$a_{N,\ldots,0}$	1.00000	0.92931	1.56646	0.83918	0.43723	
5	$b_{N,\ldots,0}$	0.11635	0	0.39623	0	0.31327	1.0955
	$a_{N,\ldots,0}$	0.92004	1.93449	1.22581	0.89778	0.31327	
6	$b_{N,\ldots,0}$	0.26420	0	0.50595	0.03163 0	0 0.27874	1.0380
	$a_{N,\ldots,0}$	2.37658	1.58926	1.68344	1.00000 0.67748	0.91055 0.31275	

8.5 Elliptic Design

Passband Ripple $\epsilon_{\text{dB}} = 1$, Stopband Attenuation $A_s = 40$ dB

N	$H(s) = \frac{b_N s^N + b_{N-1} s^{N-1} + \ldots + b_1 s + b_0}{a_N s^N + a_{N-1} s^{N-1} + \ldots + a_1 s + a_0}$						$\frac{\Omega_s}{\Omega_p}$
2	$b_{N,\ldots,0}$			0.01001	0	0.98757	7.0434
	$a_{N,\ldots,0}$			1.00000	1.09150	1.10807	
3	$b_{N,\ldots,0}$		0	0.06920	0	0.52652	2.4162
	$a_{N,\ldots,0}$		1.00000	0.97825	1.24339	0.52652	
4	$b_{N,\ldots,0}$	0.01000	0	0.15020	0	0.32196	1.5154
	$a_{N,\ldots,0}$	1.00000	0.93913	1.51372	0.80369	0.36125	
5	$b_{N,\ldots,0}$	0.04698	0	0.22007	0	0 0.22985	1.2187
	$a_{N,\ldots,0}$	0.92339	1.84712	1.12923	0.78813	1.00000 0.22985	
6	$b_{N,\ldots,0}$	0.11725	0	0.27999	0.01000 0	0 0.18600	1.0989
	$a_{N,\ldots,0}$	2.23780	1.47986	1.43164	1.00000 0.56516	0.91544 0.20869	

8.5 Elliptic Design

- These tables assume quadratic filter sections described above have been written as a ratio of N-order polynomials, where the numerator contains only even-order coefficients due to the zeros being on the $j\omega$-axis, e.g.,

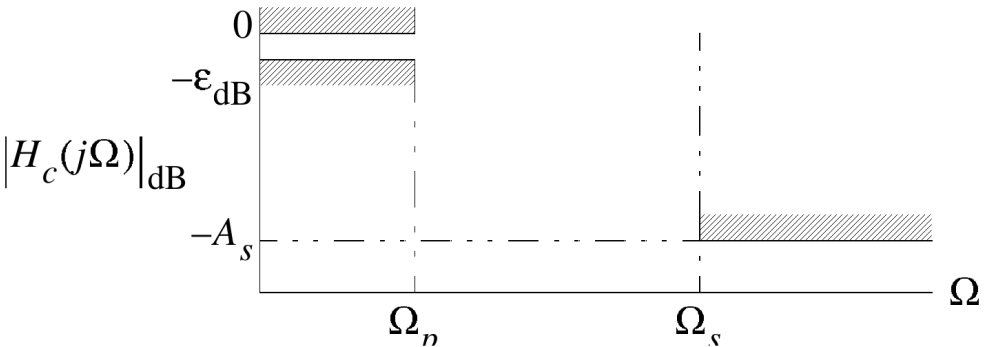
$$H(s) = \begin{cases} \dfrac{b_N s^N + b_{N-2} s^{N-2} + \cdots + b_2 s^2 + b_0}{s^N + a_{N-1} s^{N-1} + \cdots + a_1 s + a_0}, & N \text{ even}, \\ \dfrac{b_{N-1} s^{N-1} + b_{N-3} s^{N-3} + \cdots + b_2 s^2 + b_0}{s^N + a_{N-1} s^{N-1} + \cdots + a_1 s + a_0}, & N \text{ odd}. \end{cases}$$

Also the filters have a normalized cutoff frequency of $\Omega_p = 1 \ \mathrm{rad/s}$.

8.5.1 Elliptic Design from Amplitude Specifications

- Given ϵ_{dB}, A_s, Ω_p, and Ω_s, compute the transition ratio $\Omega_r = \Omega_s/\Omega_p$.
- Choose the smallest order N whose tabulated Ω_r is less than or equal to the desired ratio.
- Frequency scaling is then performed with $s \mapsto s/\Omega_p$.

8.5.1 Elliptic Design from Amplitude Specifications



Amplitude-response constraints.

Example 8.7 – Elliptic Lowpass Design

- Specifications: $\epsilon_{\mathrm{dB}} = 1$ dB, $A_s = 40$ dB, $f_p = 10$ kHz, and $f_s = 14.4$ kHz.
- The desired transition ratio is $\Omega_r = 14.4/10 = 1.44$.
- The 1 dB / 40 dB table shows that order $N = 5$ meets this requirement.
- The normalized design is

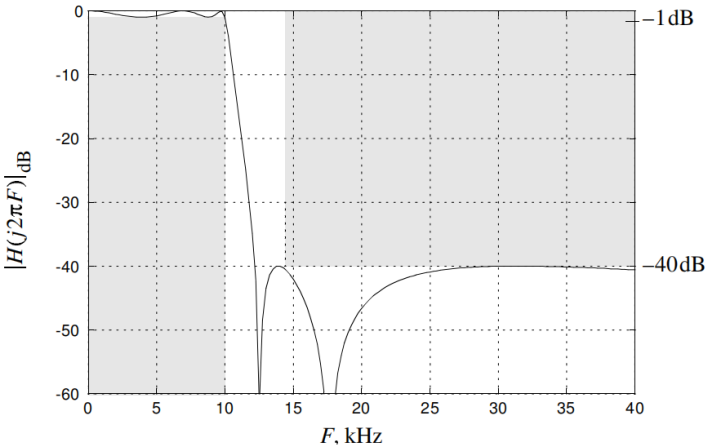
$$H(s) = \frac{0.04698s^4 + 0.22007s^2 + 0.22985}{s^5 + 0.92339s^4 + 1.84712s^3 + 1.12923s^2 + 0.78813s + 0.22985},$$

  followed by the scaling $s \mapsto s/(2\pi \cdot 10{,}000)$.
- MATLAB verification and synthesis use `freqs`, `ellipord`, and `ellip`.

Example 8.7 – MATLAB Elliptic Design Commands

```
f = 0:50000/200:50000;  w = 2*pi*f;
H = freqs([.04698 0 .22007 0 .22985], ...
          [1 .92339 1.84712 1.12923 .78813 .22985], ...
          w/(2*pi*10000));
plot(w/(1000*2*pi),20*log10(abs(H)))
[n,Wn] = ellipord(2*pi*10000,2*pi*14400,1,40,'s')
[b,a]  = ellip(n,1,40,Wn,'s')
```

Example 8.7 – Elliptic Magnitude Response



Elliptic magnitude response.

8.6 The Design of IIR Filters from Analog Prototypes

- This section maps classical continuous-time filter designs into discrete-time IIR filters.
- The two principal methods are impulse invariance and the bilinear transformation.
- Impulse invariance preserves sampled impulse-response behavior but introduces aliasing.
- The bilinear transform avoids aliasing by using a one-to-one mapping between the s-plane and the z-plane, at the cost of frequency warping.

8.6.1 Impulse Invariant Design

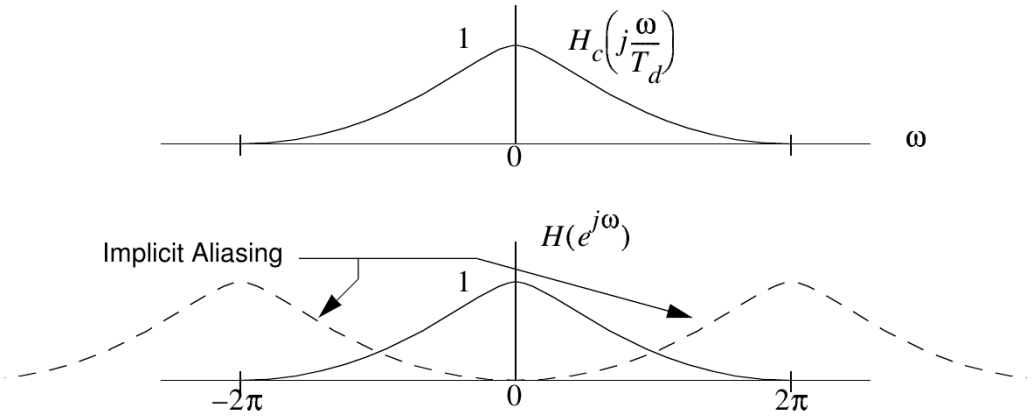
Basic idea

Choose the discrete-time impulse response as a sampled version of the continuous-time impulse response:

$$h[n] = Gh_c(nT_d), \qquad H(e^{j\omega}) = \frac{G}{T_d} \sum_{k=-\infty}^{\infty} H_c\left(j\frac{\omega}{T_d} + j\frac{2\pi k}{T_d}\right).$$

- The digital frequency response is therefore an aliased version of the analog response.
- To reduce aliasing, use lowpass prototypes with monotone stopbands and choose smaller T_d (larger sampling rate) or a lower cutoff frequency.
- G is a constant that will be specified later.

8.6.1 Impulse Invariant Design – Aliasing Concept



T_d = design sampling period

8.6.1 Impulse Invariant Design – Partial-Fraction Construction

From poles in the s-plane to poles in the z-plane

If

$$H_c(s) = \sum_{k=1}^{N} \frac{A_k}{s - s_k},$$

then

$$H(z) = G \sum_{k=1}^{N} \frac{A_k}{1 - e^{s_k T_d} z^{-1}}, \qquad p_k = e^{s_k T_d}.$$

- The gain factor G is chosen so that the dc gains match: $H(1) = H_c(0)$.
- This construction is especially convenient for rational $H_c(s)$ with distinct first-order poles.

Example 8.8 – Impulse Invariant Design from a Rational $H_c(s)$

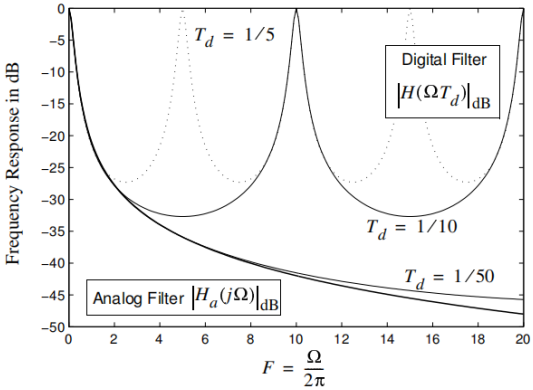
- Start with
$$H_c(s) = \frac{0.5(s+4)}{(s+1)(s+2)} = \frac{1.5}{s+1} - \frac{1}{s+2}.$$
- The inverse Laplace transform is
$$h_c(t) = \left[1.5e^{-t} - e^{-2t}\right] u(t).$$
- Sampling and gain scaling give
$$H(z) = G\left[\frac{1.5}{1 - e^{-T_d} z^{-1}} - \frac{1}{1 - e^{-2T_d} z^{-1}}\right],$$
with G selected from $H(1) = H_c(0)$.

Example 8.8 – Impulse Invariant MATLAB Synthesis

- MATLAB provides direct synthesis through `impinvar(bs,as,Fs)`.
- When the starting point is a discrete-time specification, first map the digital edge frequencies to continuous-time values with $\Omega_i = \omega_i / T_d$, design $H_c(s)$, and then apply impulse invariance.

```
as = conv([1 1],[1 2]);
bs = 0.5*[1 4];
[bz10,az10] = impinvar(bs,as,10);   % fs = 10 Hz
[bz50,az50] = impinvar(bs,as,50);   % fs = 50 Hz
[Hc,wc] = freqs(bs,as);
[H10,F10] = freqz(bz10,az10,256,'whole',10);
[H50,F50] = freqz(bz50,az50,256,'whole',50);
```

Example 8.8 – Impulse Invariant Response versus Sampling Rate



Frequency response of impulse invariant design for various T_d values

Example 8.9 – Impulse Invariant Design from Digital Specifications

Requirement: Use a Chebyshev type I design with $\epsilon_{\text{dB}} = 1$ dB, $A_s = 20$ dB, $\omega_p = 0.1\pi$, and $\omega_s = 0.3\pi$.

- Using the design formula for N,

$$N = \lceil 2.08 \rceil = 3.$$

Note:

$$\frac{\Omega_s}{\Omega_p} = \frac{\omega_s}{\omega_p}.$$

- The normalized system function is

$$H_c(s) = \frac{0.0152}{s^3 + 0.3105s^2 + 0.1222s + 0.0152},$$

with poles at

$$0.1549e^{j180^\circ}, \qquad 0.3132e^{\pm j104.4^\circ}.$$

Example 8.9 – Impulse Invariant Design from Digital Specifications

- To frequency scale $H_c(s)$, let

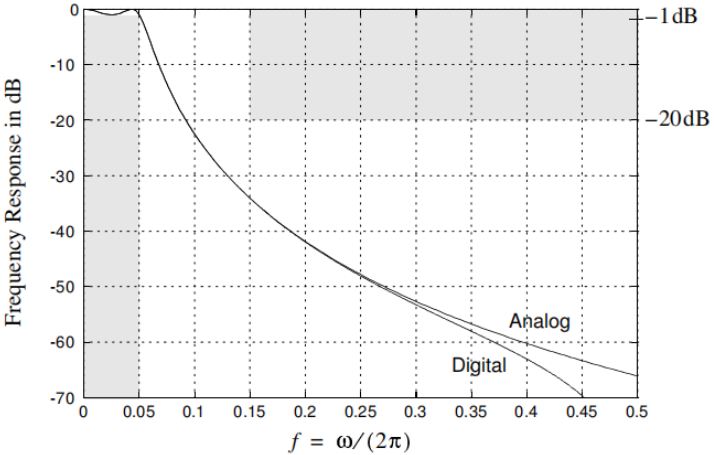
$$s \longmapsto \frac{s}{\omega_p / T_d}$$

- Calculate $H(z)$ based on $H_c(s)$ (left as an exercise).

Example 8.9 – Impulse Invariant Design from Digital Specifications

```
[n,Wn] = cheb1ord(0.1*pi,0.3*pi,1,20,'s')
[bs,as] = cheby1(n,1,Wn,'s')
W = 0:4/200:4;
Hc = freqs(bs,as,W);
[bz,az] = impinvar(bs,as,1);
[H,w] = freqz(bz,az);
plot(W/(2*pi),20*log10(abs(Hc)))
plot(w/(2*pi),20*log10(abs(H/H(1))))
```

Example 8.9 – Chebyshev Impulse Invariant Response



Frequency response of Chebyshev impulse invariant design for $f_s = 1$ Hz.

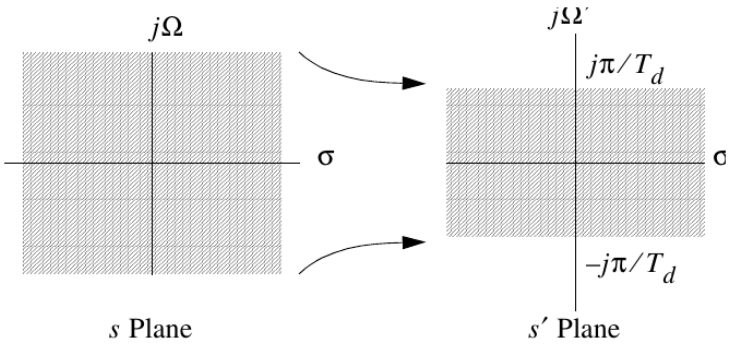
8.6.2 Bilinear Transformation Design

Bilinear transform

$$s = \frac{2}{T_d}\left(\frac{1 - z^{-1}}{1 + z^{-1}}\right), \qquad z = \frac{1 + \frac{T_d}{2}s}{1 - \frac{T_d}{2}s}.$$

- The many-to-one mapping $z = e^{sT_d}$ of impulse invariance is replaced by a one-to-one mapping that sends the entire left half of the s-plane into the interior of the unit circle.
- The price paid is frequency warping rather than aliasing.

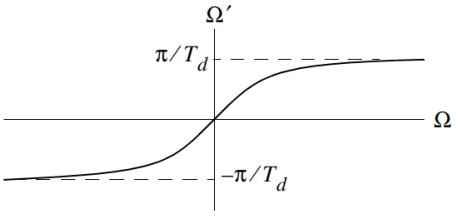
8.6.2 Bilinear Transformation Design



- The one-to-one mapping of interest is

$$s' = \frac{2}{T_d} \tanh^{-1}\left(\frac{sT_d}{2}\right).$$

8.6.2 Bilinear Transformation Design



Mapping from Ω to Ω′

- Consider just the $j\Omega$-axis for a moment:

$$\Omega' = \frac{2}{T_d} \tan^{-1}\left(\frac{\Omega T_d}{2}\right).$$

- Clearly, the entire Ω-axis has been compressed, or warped, to fit the interval

$$\left[-\frac{\pi}{T_d}, \frac{\pi}{T_d}\right].$$

8.6.2 Bilinear Transformation Design

- The complete mapping from s to z is obtained by setting

$$z = e^{s' T_d}, \qquad s' = \frac{1}{T_d} \ln z.$$

Therefore

$$s' = \frac{1}{T_d} \ln z = \frac{2}{T_d} \tanh^{-1}\left(\frac{sT_d}{2}\right),$$

or

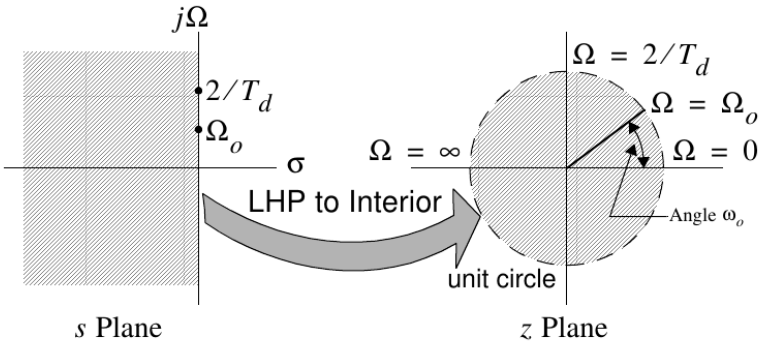
$$s = \frac{2}{T_d} \tanh\left(\frac{\ln z}{2}\right).$$

- Since

$$\tanh\left(\frac{\ln z}{2}\right) = \frac{1 - z^{-1}}{1 + z^{-1}},$$

$$s = \frac{2}{T_d}\left(\frac{1 - z^{-1}}{1 + z^{-1}}\right), \qquad z = \frac{1 + \frac{T_d}{2}s}{1 - \frac{T_d}{2}s}.$$

8.6.2 Bilinear Transformation Design



- The basic filter design equation is

$$H(z) = H_c(s)\Big|_{s=\frac{2}{T_d}\left(\frac{1-z^{-1}}{1+z^{-1}}\right)}.$$

8.6.2 Bilinear Design – Frequency Warping and Prewarping

Frequency mapping

$$\Omega = \frac{2}{T_d}\tan\left(\frac{\omega}{2}\right), \qquad \omega = 2\tan^{-1}\left(\frac{\Omega T_d}{2}\right).$$

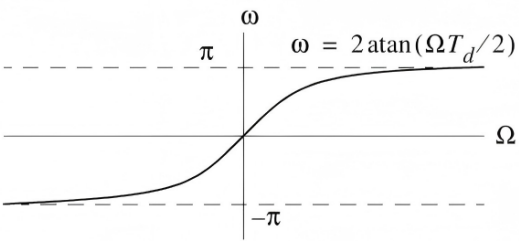
- The nonlinear relation between Ω and ω is called frequency warping.
- To preserve critical digital frequencies, prewarp the analog edge frequencies using

$$\Omega_i = \frac{2}{T_d}\tan\left(\frac{\omega_i}{2}\right).$$

- Lowpass designs that have zeros at infinity in the s-plane gain zeros at $z = -1$ after the bilinear map.

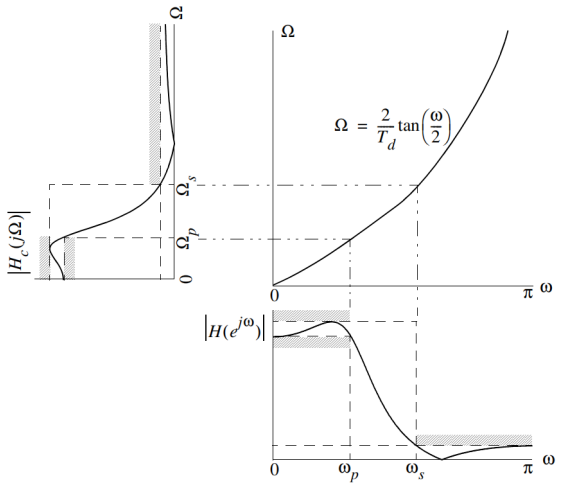
8.6.2 Bilinear Design – Mapping Ω to ω

$$\Omega = \frac{2}{T_d}\tan(\omega/2) \quad \text{or} \quad \omega = 2\tan^{-1}(\Omega T_d/2)$$



Mapping from Ω to ω.

8.6.2 Bilinear Design – Frequency Warping



Frequency-axis warping of a continuous-time lowpass prototype into the digital domain.

8.6.2 Bilinear Design – Practical Design Steps

- Given discrete-time amplitude specifications, corresponding critical frequencies, and the sampling spacing T_d, first design a corresponding continuous-time filter $H_c(s)$ using prewarped critical frequencies:

$$\Omega_p = \frac{2}{T_d} \tan\left(\frac{\omega_p}{2}\right), \qquad \Omega_s = \frac{2}{T_d} \tan\left(\frac{\omega_s}{2}\right).$$

  For convenience, T_d may be set to unity since upon transforming $H_c(s)$ back to $H(z)$, it will cancel out. The first step in finding $H_c(s)$ is to determine N and any other needed parameters.

- Map the $H_c(s)$ design to $H(z)$ using the bilinear transform:

$$H(z) = H_c\left(\frac{2}{T_d}\frac{1-z^{-1}}{1+z^{-1}}\right).$$

Example 8.10 – Bilinear Lowpass Design

Design a discrete-time lowpass filter to satisfy the following amplitude specifications:

$$\epsilon_{\mathrm{dB}} = 3.01\,\mathrm{dB}, \qquad A_s = 15\,\mathrm{dB}, \qquad \omega_p = 0.5\pi, \qquad \omega_s = 0.75\pi.$$

Assume a monotonic passband and stopband is desired. Also assume $\frac{1}{T_d} = f_s = 2\,\mathrm{kHz}$.

- The prewarped critical frequencies are

$$\Omega_p = 2 \cdot 2000 \tan(0.5\pi/2) = 4000\ \mathrm{rad/s},$$

and

$$\Omega_s = 2 \cdot 2000 \tan(0.75\pi/2) = 9657\ \mathrm{rad/s}.$$

- Since both the passband and stopband are required to be monotonic, a Butterworth approximation will be used.

Example 8.10 – Bilinear Lowpass Design

- The Butterworth order is

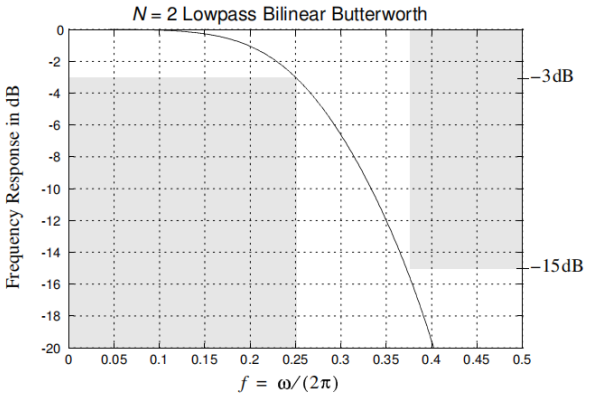
$$N = \left\lceil \frac{\log_{10}\left[\dfrac{10^{3.01/10}-1}{10^{15/10}-1}\right]}{2\log_{10}(4000/9657)} \right\rceil = \lceil 1.9412 \rceil = 2.$$

- From the Butterworth design tables, we can immediately write

$$H_c(s) = \left. \frac{1}{s^2+\sqrt{2}s+1} \right|_{s \mapsto s/4000}.$$

- Using $s = \frac{2}{T_d}\left(\frac{1-z^{-1}}{1+z^{-1}}\right)$, we can obtain $H(z)$.

Example 8.10 – Magnitude Response



Magnitude response in dB from MATLAB analysis.

Example 8.11 – Bilinear Transform Applied to a Rational $H_c(s)$

- Revisit Example 8.8 with the analog specification

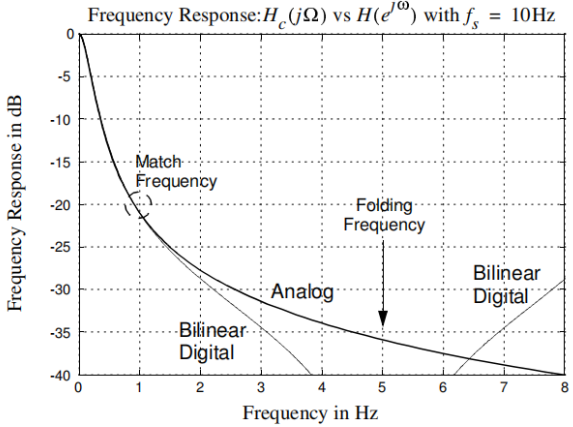
$$H_c(s) = \frac{0.5(s+4)}{(s+1)(s+2)}.$$

- Choose $f_s = 10$ Hz and a matching frequency of 1 Hz.
- MATLAB uses `bilinear(bs,as,10,1)` to create the z-domain model and compare it against the analog response.
- The response matches exactly at the designated matching frequency, while the bilinear design remains alias-free over the digital band.

Example 8.11 – Bilinear Design Commands

```
as = conv([1 1],[1 2]);
bs = 0.5*[1 4];
[bz,az] = bilinear(bs,as,10,1)
[Hc,wc] = freqs(bs,as);
[H10,F10] = freqz(bz,az,512,'whole',10);
plot(wc/(2*pi),20*log10(abs(Hc)))
plot(F10,20*log10(abs(H10)),'r')
axis([0 8 -40 0])
```

Example 8.11 – Bilinear Design Response



Magnitude response in dB from the MATLAB bilinear design.

8.7 Frequency Transformations – Overview

- The classical lowpass prototypes developed earlier can be transformed into highpass, bandpass, and bandstop filters.
- If the final design is discrete time, two paths are available:
  - transform the analog prototype first and then discretize, or
  - discretize the lowpass design first and then apply a discrete-time frequency transformation.
- With the bilinear transform, either continuous-time or discrete-time transformations are acceptable.

8.7.1 Continuous-Time Transformations – Lowpass and Highpass

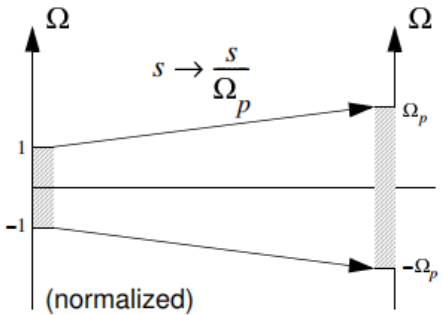
- The lowpass prototype is assumed normalized to a 1 rad/s cutoff.
- Lowpass scaling is simply

$$s \mapsto \frac{s}{\Omega_p}, \qquad H_{\text{desired}}(s) = H_c\left(\frac{s}{\Omega_p}\right).$$

- The lowpass-to-highpass transformation is

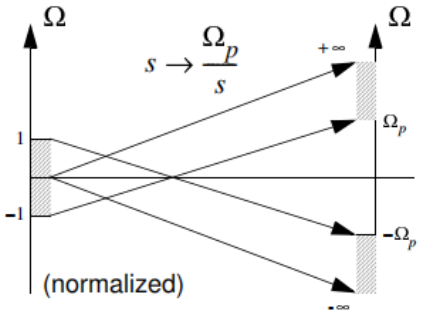
$$s \mapsto \frac{\Omega_p}{s}, \qquad H_{\text{desired}}(s) = H_c\left(\frac{\Omega_p}{s}\right).$$

8.7.1 Continuous-Time Lowpass Mapping



Lowpass to lowpass mapping.

8.7.1 Continuous-Time Highpass Mapping



Lowpass to highpass mapping.

8.7.1 Continuous-Time Transformations – Bandpass and Bandstop

- For desired lower and upper passband edges Ω_{p1} and Ω_{p2}, define

$$\Omega_c = \sqrt{\Omega_{p1}\Omega_{p2}}, \qquad \Omega_b = \Omega_{p2} - \Omega_{p1}.$$

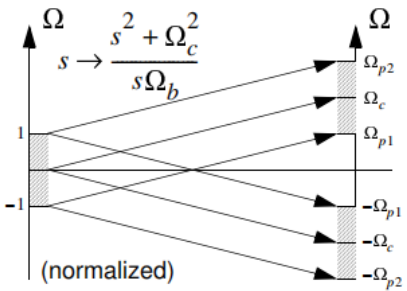
- Lowpass to bandpass:

$$s \mapsto \frac{s^2 + \Omega_c^2}{s\Omega_b}.$$

- Lowpass to bandstop:

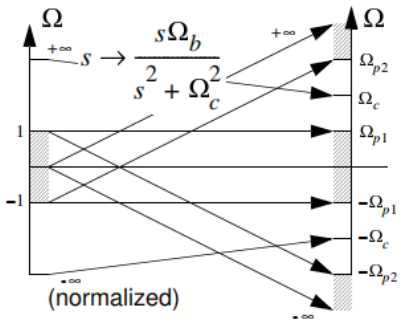
$$s \mapsto \frac{s\Omega_b}{s^2 + \Omega_c^2}.$$

8.7.1 Continuous-Time Bandpass Mapping



Lowpass to bandpass mapping.

8.7.1 Continuous-Time Bandstop Mapping



Lowpass to bandstop mapping.

Example 8.12 – Bandpass Bilinear Design

- Specifications: $f_s = 200$ kHz, $\epsilon_{\text{dB}} = 1$ dB, $A_s = 20$ dB, $f_{s1} = 20$ Hz, $f_{p1} = 50$ Hz, $f_{p2} = 20$ kHz, and $f_{s2} = 45$ kHz.
- Prewarped analog frequencies are

  $$\Omega_{s1} = 125.66\ \text{rad/s}, \quad \Omega_{s2} = 341.63\ \text{krad/s}, \quad \Omega_{p1} = 314.16\ \text{rad/s}, \quad \Omega_{p2} = 129.67\ \text{krad/s}.$$

- Mapping the bandpass problem to a lowpass prototype gives two candidate stopband values,

  $$A = 2.5052, \qquad B = 2.6401,$$

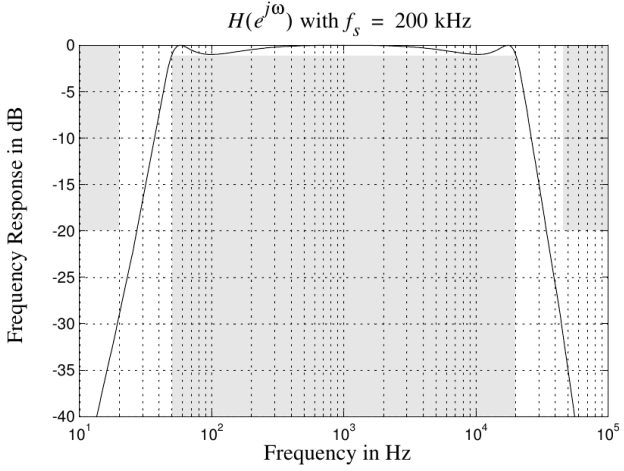
  so $\Omega_s = \min\{A, B\} = 2.5052$.
- The Chebyshev type I order is $N = 3$, and the normalized prototype is

$$H_c(s) = \frac{0.4913}{s^3 + 0.988s^2 + 1.238s + 0.491}.$$

Example 8.12 – Bilinear Bandpass MATLAB Analysis

```
F  = logspace(1,5,500);
f  = F/200000;
s1 = (1-exp(-j*2*pi*f))./(1+exp(-j*2*pi*f));
s  = ((2*200000*s1).^2 + wc^2)./(2*200000*s1*wb);
H  = 0.4913./(s.^3 + 0.988*s.^2 + 1.238*s + 0.491);
semilogx(F,20*log10(abs(H)))
axis([10 100000 -40 2]); grid
```

Example 8.12 – Bilinear Bandpass Response

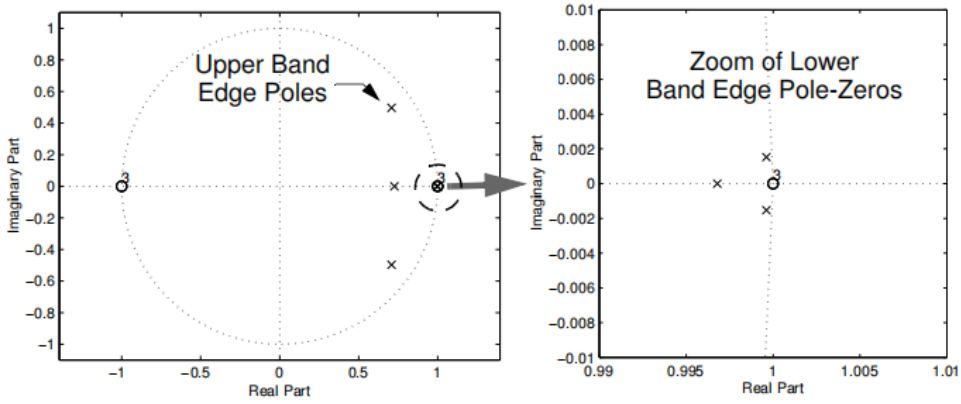


Bilinear bandpass response.

Example 8.12 – Complete MATLAB Design

```
[n,Wn] = cheb1ord([50/100000 20000/100000], ...
                  [20/100000 45000/100000],1,20);
[b,a] = cheby1(n,1,Wn);
zplane(b,a)
[H,F] = freqz(b,a,512,200000);
semilogx(F,20*log10(abs(H)))
grid; axis([10 100000 -40 0])
```

Example 8.12 – z-Domain Pole-Zero Plot



z-domain pole-zero plot.

8.8 FIR Filters – Basic Characteristics

- FIR filters have finite-duration impulse responses and are also called nonrecursive, moving-average, transversal, or tapped-delay-line filters.
- Advantages over IIR filters:
  - exact linear phase is possible,
  - nonrecursive realizations are inherently stable,
  - coefficient quantization effects are easier to control.
- Disadvantages:
  - much higher order is usually required for the same amplitude selectivity,
  - computational complexity increases,
  - memory requirements are larger.

8.8.1 Design Using Windowing

- Let the desired response be

$$H_d(e^{j\omega}) = \sum_{n=-\infty}^{\infty} h_d[n] e^{-j\omega n}.$$

- A simple FIR approximation truncates the ideal impulse response to

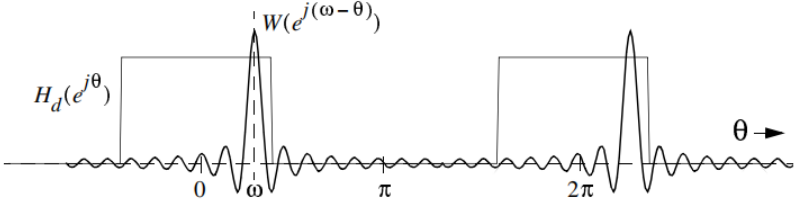
$$h[n] = \begin{cases} h_d[n], & 0 \le n \le M, \\ 0, & \text{otherwise.} \end{cases}$$

- More generally use a window,

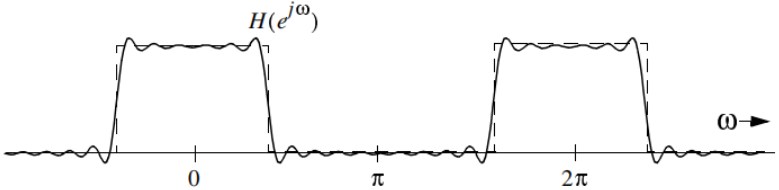
$$h[n] = h_d[n] w[n],$$

  to soften the abrupt truncation and reduce Gibbs oscillations.
- In frequency, this corresponds to circular convolution of $H_d(e^{j\omega})$ with the window transform $W(e^{j\omega})$.

8.8.1 Windowing – Circular Convolution Interpretation



(a) Circular convolution (windowing)



(b) Resulting frequency response of windowed sinc(x)

8.8.1 Windowing – Imposing Linear Phase

- If the window is symmetric about $M/2$ and the desired impulse response is symmetric or antisymmetric about $M/2$, then the FIR approximation has generalized linear phase.
- For the symmetric case,

$$H(e^{j\omega}) = A_e(e^{j\omega})e^{-j\omega M/2},$$

  with $A_e(e^{j\omega})$ real and even.
- For the antisymmetric case,

$$H(e^{j\omega}) = jA_o(e^{j\omega})e^{-j\omega M/2},$$

  with $A_o(e^{j\omega})$ real and odd.

8.8.2 Lowpass FIR Design

Ideal lowpass target

$$H_d(e^{j\omega}) = \begin{cases} e^{-j\omega M/2}, & |\omega| < \omega_c, \\ 0, & \text{otherwise}, \end{cases} \qquad h_d[n] = \frac{\sin(\omega_c(n - M/2))}{\pi(n - M/2)}.$$

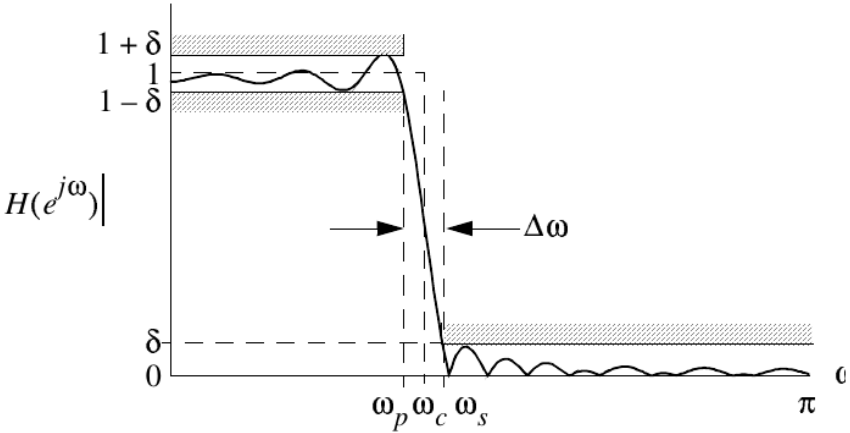
- With a symmetric window, the actual FIR coefficients become $h[n] = h_d[n]w[n]$.
- The usual specifications are passband edge ω_p, stopband edge ω_s, and stopband ripple δ.
- The corresponding attenuation measures are $A_s = 20\log_{10}\delta$ and $\epsilon_{\mathrm{dB}} = 20\log_{10}(1+\delta)$.

8.8.2 Window Characteristics for FIR Design

Window	Transition width	Min. stopband A_s	Equivalent Kaiser β
Rectangular	$1.81\pi/M$	21 dB	0.00
Bartlett	$1.80\pi/M$	25 dB	1.33
Hanning	$5.01\pi/M$	44 dB	3.86
Hamming	$6.27\pi/M$	53 dB	4.86
Blackman	$9.19\pi/M$	74 dB	7.04

- Choose the window that just meets the stopband requirement.
- Choose M so that $\Delta\omega \leq \omega_s - \omega_p$ and then set $\omega_c = (\omega_p + \omega_s)/2$.

8.8.2 Lowpass FIR Specifications



Lowpass FIR amplitude specifications.

8.8.2 Lowpass FIR Design – Window and Kaiser Methods

- Standard windows trade transition width against minimum stopband attenuation. The notes tabulate this tradeoff for the rectangular, Bartlett, Hanning, Hamming, and Blackman windows.
- General steps: choose a window that meets A_s, choose M so that the transition-width requirement is satisfied, set $\omega_c = (\omega_p + \omega_s)/2$, and then verify and adjust if necessary.
- The Kaiser window offers a one-parameter design:

$$w[n] = \frac{I_0\left(\beta\sqrt{1 - [(n-\alpha)/\alpha]^2}\right)}{I_0(\beta)}, \qquad \alpha = M/2.$$

8.8.2 Lowpass FIR Design – Window and Kaiser Methods

- Approximate design rules are

$$\beta \approx \begin{cases} 0, & A_s < 21, \\ 0.5842(A_s - 21)^{0.4} + 0.07886(A_s - 21), & 21 \le A_s \le 50, \\ 0.1102(A_s - 8.7), & A_s > 50, \end{cases}$$

and

$$M \approx \frac{A_s - 8}{2.285\,\Delta\omega}.$$

Example 8.13 – FIR Lowpass by Windowing

- Specifications: $\omega_p = 0.4\pi$, $\omega_s = 0.6\pi$, and $\delta = 0.0032$ so that $A_s = 50$ dB.
- From the tabulated window data, a Hamming window is sufficient.
- The transition-width requirement gives

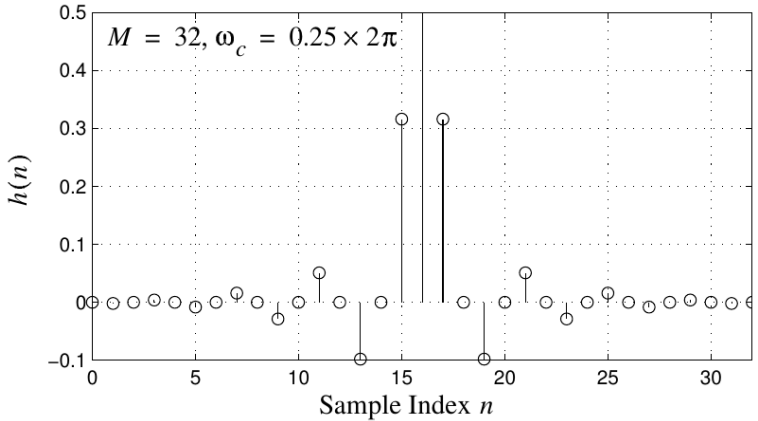
$$0.1\pi \geq \frac{6.27\pi}{M} \quad \Rightarrow \quad M \geq 32.$$

- The cutoff is chosen as $\omega_c = (\omega_p + \omega_s)/2 = 0.5\pi = 0.25 \times 2\pi$.
- For a Kaiser design, $\beta \approx 4.5335$ and the approximate length is $M \approx 30$.
- MATLAB verification uses `fir1` with Hamming or Kaiser windows.

Example 8.13 – MATLAB FIR Design Commands

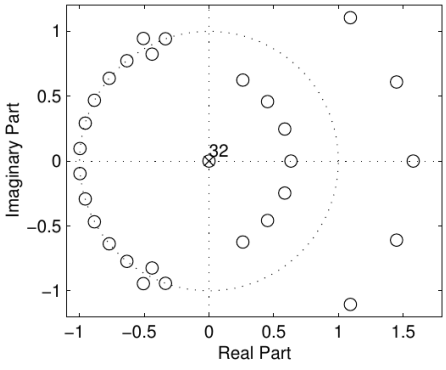
```
% Hamming window design
b  = fir1(32,2*.25,hamming(32+1));
stem(0:length(b)-1,b); grid
zplane(b,1)
[H,F] = freqz(b,1,512,1);
plot(F,20*log10(abs(H)))
axis([0 .5 -70 2]); grid

% Kaiser window design
bk = fir1(30,2*.25,kaiser(30+1,4.5335));
[Hk,F] = freqz(bk,1,512,1);
plot(F,20*log10(abs(Hk)))
axis([0 .5 -70 2]); grid
```

Example 8.13 – Hamming Impulse Response



$M = 32$, Hamming window impulse response.

Example 8.13 – Hamming Pole-Zero Plot

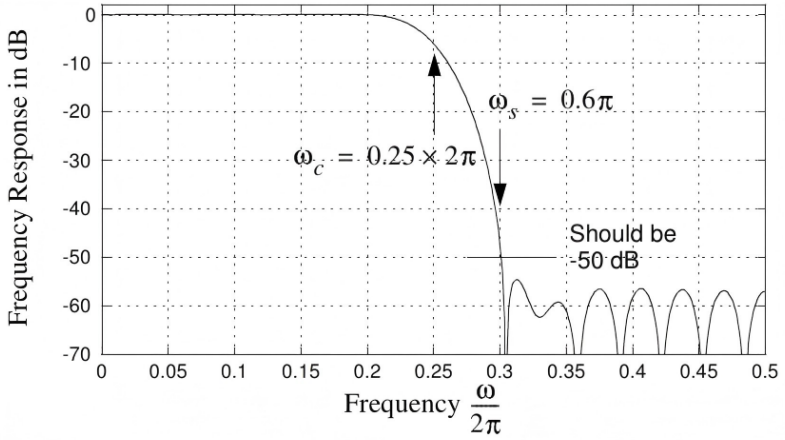


Note: There are few misplaced zeros in this plot due to MATLAB's problem finding the roots of a large polynomial.

Note: The many zero quadruplets that appear in this linear phase design.

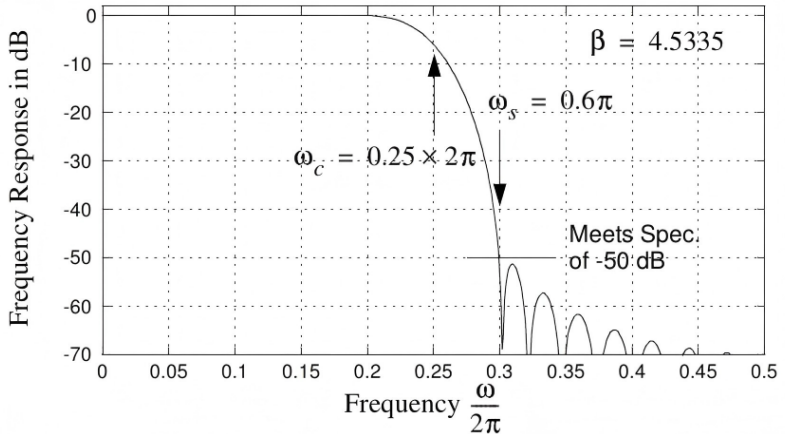
$M = 32$, Hamming window pole-zero plot.

Example 8.13 – Hamming Frequency Response



$M = 32$, Hamming window frequency response.

Example 8.13 – Kaiser Frequency Response



$M = 30$, Kaiser window ($\beta = 4.5335$) frequency response.

8.9 MATLAB Functions – Introduction and Analysis Functions

- The signal-processing toolbox collects the functions needed to analyze, design, discretize, and realize filters in both the s-domain and the z-domain.
- Useful analysis / implementation functions include `filter`, `freqs`, `freqz`, `grpdelay`, `impz`, `unwrap`, and `zplane`.
- Common linear-system transformation tools include `residuez`, `tf2zp`, and `zp2sos`.

8.9 MATLAB Functions – Analysis and Implementation

Function	Purpose
`y = filter(b,a,x)`	Direct-form II filtering of the vector x.
`[H,w] = freqs(b,a)`	Continuous-time (s-domain) frequency response computation.
`[H,w] = freqz(b,a)`	Discrete-time (z-domain) frequency response computation.
`[Gpd,w] = grpdelay(b,a)`	Group-delay computation.
`h = impz(b,a)`	Impulse-response computation.
`unwrap()`	Phase unwrapping.
`zplane(b,a)`	Plotting of the z-plane pole-zero map.

8.9 MATLAB Functions – Linear Transformations and IIR Design

Function	Purpose
`residuez()`	z-domain partial-fraction conversion.
`tf2zp()`	Transfer-function to zero-pole conversion.
`zp2sos()`	Zero-pole to second-order (biquadratic) sections conversion.
`[b,a] = besself(n,Wn)`	Bessel analog filter design; near-constant group delay in the analog domain.
`[b,a] = butter(n,Wn,'ftype','s')`	Butterworth analog or digital design; `ftype` selects bandpass, high-pass, or bandstop.
`[b,a] = cheby1(n,Rp,Wn,'ftype','s')`	Chebyshev type I analog or digital design.
`[b,a] = cheby2(n,Rs,Wn,'ftype','s')`	Chebyshev type II analog or digital design.
`[b,a] = ellip(n,Rp,Rs,Wn,'ftype','s')`	Elliptic analog or digital design.

8.10 MATLAB Functions – Order Selection and FIR Design

Function	Purpose
`[n,Wn] = buttord(Wp,Ws,Rp,Rs,'s')`	Butterworth analog or digital order selection.
`[n,Wn] = cheb1ord(Wp,Ws,Rp,Rs,'s')`	Chebyshev type I order selection.
`[n,Wn] = cheb2ord(Wp,Ws,Rp,Rs,'s')`	Chebyshev type II order selection.
`[n,Wn] = ellipord(Wp,Ws,Rp,Rs,'s')`	Elliptic filter order selection.
`fir1()`	Window-based FIR design with standard responses.
`fir2()`	Window-based FIR design with arbitrary responses.
`remez()`	Parks–McClellan optimal FIR design.
`remezord()`	Parks–McClellan order estimation.

8.9 MATLAB Functions – Filter Discretization

Function	Purpose
`[bz,az] = bilinear(bs,as,Fs,Fp)`	Maps an *s*-domain transfer function to the *z*-domain using the bilinear transformation. *Fs* is the sampling rate in Hz; the optional *Fp* is a matching frequency in Hz.
`[bz,az] = impinvar(bs,as,Fs)`	Maps an *s*-domain transfer function to the *z*-domain using impulse invariance. *Fs* is the sampling rate in Hz.

8.9 MATLAB Functions – Order Selection, and Discretization

- IIR design functions include `besself`, `butter`, `cheby1`, `cheby2`, and `ellip`.
- Order-selection functions include `buttord`, `cheb1ord`, `cheb2ord`, and `ellipord`.
- FIR design functions include `fir1`, `fir2`, `remez`, and `remezord`.
- Discretization from the analog domain is handled by `bilinear` and `impinvar`.